LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

CVE-2026-111497 Node2Shell — Brotli Decoder Remote Code Execution

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-10-27
Version	1.0
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-111497 Node2Shell — Brotli Decoder Remote Code Execution - CVE / Vulnerability Threat Intelligence Report

Published: 2026-10-27 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
    7.1. Exposure-Discovery Commands
8. Mitigation and Workarounds
9. Detection Engineering
    9.1. Detection Strategy
    9.2. Sigma Rule — Candidate External Brotli Input
    9.3. Microsoft Defender XDR / Sentinel Hunt — Node Process Anomalies
    9.4. Splunk Hunt — Brotli Requests Followed by Failure or Restart
10. Threat Hunting
    10.1. Hypothesis 1 — Exposed Node services accepted Brotli-compressed external data
    10.2. Hypothesis 2 — Node process failures coincide with suspicious client requests
    10.3. Hypothesis 3 — A Node workload launched an unexpected child process after a suspected request
    10.4. Evidence Collection Checklist
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2026-111497 Node2Shell — Brotli Decoder Remote Code Execution

1. Executive Summary

CVE-2026-111497, commonly referred to as **Node2Shell**, is a high-severity memory-corruption vulnerability in the Brotli decoder used by Node.js and other consumers of Google Brotli. A remote, unauthenticated attacker who can cause a vulnerable process to decode attacker-controlled Brotli data may corrupt process memory. The documented impacts are denial of service, process-memory disclosure, and potential remote code execution.

The urgent operational question is not simply whether an organization runs Node.js. Risk is concentrated where a Node.js service directly invokes Brotli decompression on attacker-controlled bytes, including explicit use of `zlib.brotliDecompress*`, streaming decompression, middleware, gateways, worker services, or custom protocol handlers. Treat any internet-reachable endpoint that accepts Brotli-compressed request data and routes it to a vulnerable Node process as priority exposure until code-path and version validation are complete.

Patch Node.js to **22.23.4**, **24.22.1**, or **26.12.1** (or later in the respective supported release line). For standalone Google Brotli, remediate versions **1.0.3 through 1.2.0** with a supported patched release: 1.0.10, 1.1.1, or 1.2.1. No authoritative evidence of CISA KEV inclusion or active exploitation was identified during this review as of 2026-10-27 UTC. That absence must not delay remediation: public disclosure and a network-reachable precondition create a practical scanning and exploitation risk.

2. Vulnerability Metadata

Field	Value
CVE	CVE-2026-111497
Common name	Node2Shell
Component	Google Brotli decoder (`c/dec/decode.c`); bundled/vendored by affected consumers including Node.js
Weakness	Error condition is treated as “need more input,” allowing an internal decoder buffer to be overrun; downstream advisory maps CWE-252, Unchecked Return Value
Node.js vendor severity	High
Downstream CVSS context	GitHub-reviewed Python advisory reports CVSS v4.0 9.2: `CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`
Upstream Brotli affected range	1.0.3 through 1.2.0
Fixed upstream Brotli releases	1.0.10, 1.1.1, or 1.2.1, according to the advisory/release channel applicable to the branch
Fixed Node.js releases	22.23.4 LTS, 24.22.1 LTS, 26.12.1 Current
Disclosure / patched-release date	2026-10-21; Ubuntu CVE publication 2026-10-23
Exploitation status	No authoritative confirmation of active exploitation found in this review; assume rapid opportunistic scanning is likely after disclosure

3. Source Review & Confidence

Source	Type	Key contribution	Confidence
Node.js October 2026 security release	Primary vendor advisory	Root cause, impact, Node exposure condition, severity, affected lines	High
Node.js 22.23.4 / 24.22.1 / 26.12.1 release notes	Primary vendor release evidence	Exact fixed Node versions and dependency update	High
Google Brotli v1.2.1 release	Primary upstream release evidence	Security release and RCE impact statement	High
Ubuntu Security CVE record	Distribution advisory	Upstream range, fixed package builds, packaging caveat	High
GitHub Advisory GHSA-9v8h-qqgm-4m35	Reviewed downstream advisory	PyPI package ranges and CVSS context	Medium

The report relies on official vendor/upstream sources for technical claims. It does not reproduce malformed Brotli inputs or incorporate unverified proof-of-concept claims. See `sources.md` for the evidence record and advisory-ID caveat.

4. Vulnerability Details

The Node.js Project states that an invalid Brotli block switch was incorrectly reported as a temporary “need more input” condition rather than a permanent format error. In the streaming decoder, that behavior permits continued input accumulation into an internal **eight-byte buffer**, producing an out-of-bounds write. The failure is reached only when an application decompresses attacker-supplied Brotli bytes using a vulnerable path such as `zlib.brotliDecompress` or `zlib.createBrotliDecompress`.

This is a decoder flaw, not an HTTP-header parsing issue. `Content-Encoding: br` on a response is **not** by itself an exploit condition. Exposure requires that untrusted compressed bytes reach a vulnerable decoder. Conversely, organizations should not limit scoping to web servers: queues, RPC services, upload pipelines, serverless workers, reverse proxies that transform request bodies, and internal APIs can all provide a reachable decoding path.

5. Attack Scenarios

Scenario	Preconditions	Likely outcome	Defender focus
Public API accepts Brotli-compressed request bodies	External attacker can submit data; application or middleware decodes it in Node	Process crash, memory disclosure, or potential code execution	Patch; inspect reverse-proxy/app request telemetry and crash history
Service explicitly calls Node Brotli APIs	Application code passes user-supplied file/message/body bytes to Brotli decoder	Same memory-corruption impact, potentially outside HTTP logging coverage	Code search, dependency/version inventory, EDR process telemetry
Internal event or worker pipeline	Untrusted or laterally controlled producer can deliver compressed messages	Lateral movement / internal service compromise	Validate producer trust boundary; restrict inputs; patch worker images
Non-Node consumer bundles vulnerable Brotli	Product includes affected library or Python `brotli` / `brotlicffi` package	Product-specific impact depends on exposed decode path	Follow vendor-specific advisories; do not assume Node patch remediates all consumers

6. Threat Landscape & Exploitation Status

CVE-2026-111497 has an attractive attacker profile: remote reachability may be possible where an exposed service accepts compressed input, exploitation requires no credentials or user interaction under the published CVSS context, and successful decoder corruption can yield more than availability impact. The Attack Requirements value is **Present**, meaning a reachable application-specific decode path is required; this constrains universal exploitation but does not materially reduce urgency for exposed services.

As of the review cutoff, this assessment did **not** identify a CISA KEV listing or authoritative confirmation of exploitation in the wild. Public PoC repositories, blog claims, and scanner claims should be handled as secondary intelligence unless independently validated. Log this status as **unconfirmed**, not “safe.” Preserve relevant logs before patching because a restart may erase valuable crash and service telemetry.

7. Affected Products and Exposure

Asset / deployment pattern	Exposure assessment	Required action
Node.js 22.x earlier than 22.23.4	Affected when untrusted Brotli decoding is reachable	Upgrade to 22.23.4 or later supported release
Node.js 24.x earlier than 24.22.1	Affected when untrusted Brotli decoding is reachable	Upgrade to 24.22.1 or later supported release
Node.js 26.x earlier than 26.12.1	Affected when untrusted Brotli decoding is reachable	Upgrade to 26.12.1 or later supported release
End-of-life Node lines or custom builds	Treat as potentially affected if bundled Brotli predates the fix	Migrate to supported Node or backport only through a validated vendor process
Google Brotli 1.0.3–1.2.0	Affected library range	Upgrade to a fixed branch release (1.0.10, 1.1.1, or 1.2.1)
Python `brotli` / `brotlicffi`	GHSA identifies affected ranges before 1.2.1 / 1.2.1.0	Upgrade package and validate runtime image rebuild
Ubuntu Node.js packaging	Ubuntu notes Jammy and later use system Brotli; Focal Node.js uses vendored Brotli	Follow Ubuntu package status; do not extrapolate to non-Ubuntu builds

7.1. Exposure-Discovery Commands

Use inventory and code review first. These commands are non-exploitative and should be run in authorized environments.

```
node --version
node -p "process.versions.brotli"
node -e "const z=require('node:zlib'); console.log({brotliDecompress:!!z.brotliDecompress,
createBrotliDecompress:!!z.createBrotliDecompress})"
```

```
# Search application source and deployed build context for explicit Node Brotli decoding.
git grep -nE 'brotliDecompress|createBrotliDecompress|BrotliDecompress' -- ':!node_modules'
find /srv /opt /app -type f \( -name 'package.json' -o -name 'package-lock.json' -o -name 'yarn.lock'
\) -print 2>/dev/null
```

8. Mitigation and Workarounds

Priority	Owner	Action	Verification
Critical	Platform / application owners	Upgrade Node to 22.23.4, 24.22.1, 26.12.1, or later supported release	Record `node --version`; redeploy immutable image; restart only through change control

Critical	Product owners	Patch standalone Brotli and downstream language bindings through supported vendor packages	Package/SBOM scan confirms no affected version remains
High	Application engineering	Identify every attacker-controlled Brotli decode path and remove decoding where unnecessary	Code-review evidence; endpoint ownership assigned
High	Edge engineering	Reject or tightly size-limit compressed request bodies at the reverse proxy/WAF when application support is not required	Negative/positive functional tests; exception list documented
High	SOC / IR	Preserve application, reverse proxy, container, and crash telemetry before rollout	Retention and collection validated
Medium	Detection engineering	Deploy the included triage analytic, baselined to legitimate Brotli API clients	Alert review confirms expected false-positive rate
Medium	Vulnerability management	Add EOL Node/custom builds and vendor products with embedded Brotli to exception tracking	Asset owner, deadline, compensating control recorded

A size limit is a compensating control, not a substitute for a patch. Do not rely on HTTP `Content-Encoding` inspection alone, because applications can receive compressed data via files, queues, custom protocols, or encoded application fields.

9. Detection Engineering

9.1. Detection Strategy

There is no safe, universal network signature that proves exploitation from a Brotli blob without parsing attacker-controlled compressed content. Detection should therefore correlate three signal classes: (1) an externally sourced request or message plausibly carrying Brotli data, (2) a vulnerable Node/Brotli runtime or identified decode path, and (3) error, crash, restart, or anomalous child-process behavior near the request. This avoids falsely treating all Brotli traffic as malicious.

Signal	Telemetry	Analytic use	Caveat
`Content-Encoding: br` on POST/PUT/PATCH	Reverse proxy, WAF, API gateway	Prioritize candidate inputs for vulnerable services	Legitimate clients commonly use Brotli
Node exception, abort, segmentation fault, OOM, repeated restart	App logs, container runtime, systemd, EDR	Detect potential memory-corruption or availability effect	Not CVE-specific; correlate with requests
New shell, script interpreter, downloader, or unexpected network client spawned by Node	EDR / auditd / container runtime	Detect potential post-exploitation behavior	Requires process telemetry; apply service allowlists
Unusual egress from Node service identity	NetFlow, firewall, DNS, proxy	Identify post-compromise staging/C2 behavior	Baseline permitted service dependencies

9.2. Sigma Rule — Candidate External Brotli Input

```
title: Potential Attacker-Controlled Brotli Request to Node.js Service
id: 7f6e0f80-901d-4d21-ae20-2026111497a1
status: experimental
description: >-
  Identifies externally sourced requests carrying Brotli-compressed content to a Node.js
  application. This is triage telemetry for CVE-2026-111497, not exploit confirmation.
references:
  - https://nodejs.org/en/blog/vulnerability/october-2026-security-releases
tags:
  - attack.initial-access
  - attack.t1190
  - cve.2026.111497
logsource:
  category: webserver
  product: generic
detection:
  selection_encoding:
    http_content_encoding|contains: br
  selection_method:
    http_request_method:
      - POST
      - PUT
      - PATCH
  selection_external:
    src_ip|cidr: 0.0.0.0/0
  condition: selection_encoding and selection_method and selection_external
falsepositives:
  - Legitimate API clients sending Brotli-compressed request bodies
level: medium
```

9.3. Microsoft Defender XDR / Sentinel Hunt — Node Process Anomalies

```
let lookback = 30d;
let suspiciousChildren = dynamic(["sh", "bash", "dash", "zsh", "cmd.exe", "powershell.exe", "curl",
"wget", "python", "python3", "nc", "ncat"]);
DeviceProcessEvents
| where Timestamp > ago(lookback)
| where InitiatingProcessFileName in~ ("node", "node.exe")
| where FileName in~ (suspiciousChildren)
   or ProcessCommandLine has_any ("curl ", "wget ", "Invoke-WebRequest", "base64 -d", "/dev/tcp/")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName,
          InitiatingProcessCommandLine, FileName, ProcessCommandLine, FolderPath,
          SHA256, ReportId
| order by Timestamp desc
```

9.4. Splunk Hunt — Brotli Requests Followed by Failure or Restart

```
(index=proxy OR index=web) (http_content_encoding="*br*" OR request_headers="*Content-Encoding:*br*")
(method=POST OR method=PUT OR method=PATCH)
| stats values(uri_path) as paths values(status) as statuses count by _time, src_ip, dest, host,
service
| join type=left host [
    search index=os (process_name=node OR process_name=node.exe OR unit="*.service") ("segfault" OR
"SIGABRT" OR "heap" OR "restart" OR "out of memory")
    | stats values(_raw) as runtime_events by host
  ]
| where isnotnull(runtime_events)
| table _time, src_ip, dest, host, service, paths, statuses, runtime_events
```

Tune field names and join windows to the deployed log schema. A request-to-crash relationship is an investigation lead, not proof of successful code execution.

10. Threat Hunting

10.1. Hypothesis 1 — Exposed Node services accepted Brotli-compressed external data

```
let nodeDevices = DeviceProcessEvents
| where Timestamp > ago(30d)
| where FileName in~ ("node", "node.exe")
| distinct DeviceId, DeviceName;
DeviceNetworkEvents
| where Timestamp > ago(30d)
| where InitiatingProcessFileName in~ ("node", "node.exe")
| join kind=inner nodeDevices on DeviceId
| summarize FirstSeen=min(Timestamp), LastSeen=max(Timestamp), Connections=count(),
            RemoteIPs=make_set(RemoteIP, 50), RemoteUrls=make_set(RemoteUrl, 50)
  by DeviceName, InitiatingProcessCommandLine
| order by Connections desc
```

Purpose: establish a Node service inventory and identify externally communicating Node processes. Pivot those service names/hosts into gateway logs and locate inbound `Content-Encoding: br` requests. This avoids assuming endpoint telemetry can see HTTP headers.

10.2. Hypothesis 2 — Node process failures coincide with suspicious client requests

```
(index=proxy OR index=web) (http_content_encoding="*br*" OR request_headers="*Content-Encoding:*br*")
(method=POST OR method=PUT OR method=PATCH)
| bin _time span=5m
| stats count values(uri_path) as paths values(user_agent) as user_agents by _time, host, src_ip
| join type=left _time host [
    search index=os (process_name=node OR process_name=node.exe) ("SIGABRT" OR "segfault" OR "FATAL
ERROR" OR "heap out of memory" OR "restart")
    | bin _time span=5m
    | stats values(_raw) as node_failure_events by _time, host
  ]
| where isnotnull(node_failure_events)
| table _time, host, src_ip, paths, user_agents, node_failure_events
```

Purpose: identify a temporal relationship between candidate compressed inputs and Node runtime failure. Preserve raw events, load-balancer request IDs, container IDs, and crash dumps when policy allows.

10.3. Hypothesis 3 — A Node workload launched an unexpected child process after a suspected request

```
DeviceProcessEvents
| where Timestamp > ago(30d)
| where InitiatingProcessFileName in~ ("node", "node.exe")
| where FileName in~ ("sh", "bash", "dash", "zsh", "cmd.exe", "powershell.exe", "curl", "wget",
"python", "python3")
| summarize FirstSeen=min(Timestamp), LastSeen=max(Timestamp), Commands=make_set(ProcessCommandLine,
20)
  by DeviceName, AccountName, InitiatingProcessCommandLine, FileName
| order by LastSeen desc
```

Purpose: find potential command execution or payload retrieval from the Node process tree. Exclude known worker/process-manager behavior only after reviewing the service’s expected parent-child baseline.

10.4. Evidence Collection Checklist

Evidence	Retention / pivot
Reverse proxy/WAF access logs, including request IDs, source, method, content encoding, status, bytes, upstream	Preserve 30+ days; pivot on request ID to application logs
Node application stderr / structured logs	Search decoder errors, exceptions, uncaught exceptions, and timing anomalies
Container / systemd restart history	Correlate restarts and exit codes with ingress timestamps
EDR process and network telemetry	Examine Node child processes, service account context, and egress

	changes
Package/image SBOM	Prove version remediation; retain before/after scan results

11. MITRE ATT&CK Mapping

Technique	Name	Relevance
T1190	Exploit Public-Facing Application	Applicable when an internet-facing service decodes attacker-controlled Brotli input
T1059	Command and Scripting Interpreter	Post-exploitation hunt target when Node launches a shell or interpreter
T1105	Ingress Tool Transfer	Hunt for unexpected download clients or new external egress after suspected compromise
T1071.001	Application Layer Protocol: Web Protocols	Candidate exploitation traffic and post-compromise traffic may use HTTP(S)
T1562.001	Impair Defenses	Investigate unexpected logging disablement, agent stops, or configuration changes after a suspected foothold

This mapping is defensive context, not attribution and not evidence that any technique has occurred in a specific environment.

12. Validation / Lab Testing

No exploit payload was generated, executed, or tested. Validation completed for this report: primary-source review; version and patch-line reconciliation; safe exposure-discovery command construction; detection/hunt artifact creation; and render QA. The included analytics were not run against customer telemetry and require environment-specific field mapping, allowlisting, and validation before production alerting.

13. Recommended Actions Summary

Timeframe	Action
0–24 hours	Inventory Node/Brotli runtimes; identify externally reachable services and code paths accepting untrusted compressed data; patch supported Node lines
24–48 hours	Block unnecessary compressed request bodies at the edge; deploy candidate-input telemetry and preserve crash/restart logs
48–72 hours	Hunt for request-to-failure correlation and anomalous Node child processes; review EOL/custom builds and embedded Brotli consumers
Ongoing	Enforce SBOM-driven dependency tracking, supported-runtime lifecycle management, and regression tests for request decompression boundaries

14. References

- [1] Node.js Project — October 21, 2026 Security Releases. https://nodejs.org/en/blog/vulnerability/october-2026-security-releases

- [2] Node.js Project — Node.js 22.23.4 LTS. https://nodejs.org/en/blog/release/v22.23.4
- [3] Node.js Project — Node.js 24.22.1 LTS. https://nodejs.org/en/blog/release/v24.22.1
- [4] Node.js Project — Node.js 26.12.1 Current. https://nodejs.org/en/blog/release/v26.12.1
- [5] Google Brotli — v1.2.1 release. https://github.com/google/brotli/releases/tag/v1.2.1
- [6] Ubuntu Security — CVE-2026-111497. https://ubuntu.com/security/CVE-2026-111497
- [7] GitHub Advisory Database — GHSA-9v8h-qqgm-4m35. https://github.com/advisories/GHSA-9v8h-qqgm-4m35